

An Evaluation of Tree-Decomposition Systems for Model Counting

Xiangyu Wan

June 2023

1 Introduction

Model counting is the problem of computing the number of satisfying truth assignments of a Boolean formula, typically in the form of a CNF (conjunctive normal form). This problem has many applications including to probabilistic reasoning. There is now a yearly world-wide competition for model counters.¹

One particular approach to model counting is through compiling the Boolean formula into an equivalent tractable circuit, which is a data structure with certain properties that allows model counting to be performed in time polynomial in the circuit size. In particular, a CNF can be compiled into a smooth d-DNNF circuit which allows model counting to be done in time linear in the circuit size.² A software system called `c2d`, developed the Automated Reasoning Group at UCLA, is such a tool that converts a CNF into a d-DNNF.³ The pipeline implemented by `c2d` is as follows: first, the CNF is converted to a decomposition tree (dtree), which is a full binary tree whose leaves are in 1-to-1 correspondence with the clauses in the input CNF, through hypergraph decomposition; then the system uses information encoded by the dtree to build a smooth d-DNNF circuit from the input CNF. The quality of the dtree has a major impact on the efficiency of the compilation process and the efficiency of model counting.

We are interested in evaluating an alternative method of dtree construction for `c2d`. Specifically, constructing dtrees by conversion from tree decompositions of graphs. The construction of tree decompositions is a well studied problem in computer theory and there are a number of systems for constructing these decompositions.⁴ Our goal is to evaluate some of these systems and to see whether they improve the performance of `c2d` as far as compiling CNFs into smooth d-DNNF circuits and counting their models.

This report is structured as follows. We discuss tree decompositions in Section 2 and dtrees in Section 3. We then discuss in Section 4 the conversion

¹https://mccompetition.org/2022/mc_description.html

²d-DNNF stands for deterministic, decomposable negation normal form.

³<http://reasoning.cs.ucla.edu/c2d/>

⁴See <https://github.com/TCS-Meiji/PACE2017-TrackA> and <https://github.com/kit-algo/flow-cutter-pace17>

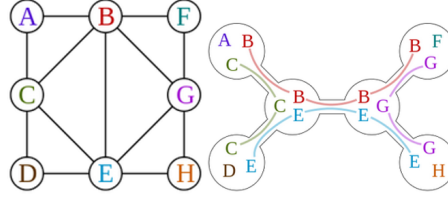


Figure 1: Graph and its tree decomposition

process from tree decompositions into dtrees. Our experimental evaluation is then presented in Section 5 followed by a conclusion in Section 6.

2 Tree Decomposition

A tree decomposition of a graph is a tree where nodes are subsets of vertices, called bags, covering all graph vertices. Edges encode relationships between bags, showing overlap or containment. The decomposition satisfies two properties: 1) each graph vertex is in at least one bag, 2) each graph edge has a bag containing its endpoints. An important notion defined in a tree decomposition is its treewidth, defined as the size of the largest bag minus one in a tree decomposition. Figure 1 shows a graph and its tree decomposition of width 2.⁵

For this project, we will explore two heuristic tree decomposition solvers, Tamaki-2017 and Flow-cutter, both of which are selected from PACE-2017’s track A, treewidth solving.

2.1 PACE Challenge

The PACE challenge is an annual competition that promotes the advancement of parameterized algorithms and complexity. In both 2016 and 2017, the PACE challenge offers a competition track for treewidth problem, which requires submissions to find a tree decomposition of the smallest treewidth possible. PACE-challenge allows both exact treewidth solvers, which run until optimal result found, and heuristic solvers, which can stop at arbitrary time to return the best result found.

2.2 Tamaki-2017

Tamaki-2017 is the submission of the team led by professor Tamaki from Meiji University. While this package implements both exact and heuristic solvers, we are only interested in its heuristic branch. It’s based on the the published work *Positive-instance driven dynamic programming for treewidth*.⁶

⁵source: https://en.wikipedia.org/wiki/Tree_decomposition

⁶<https://arxiv.org/abs/1704.05286>

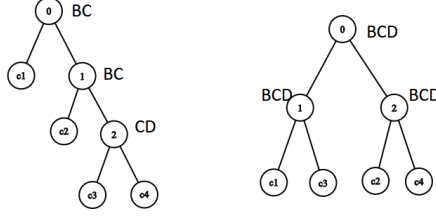


Figure 2: Two dtrees of the CNF $(\neg B \vee C) \wedge (B \vee C) \wedge (C \vee D) \wedge (D \vee E)$ where the clauses are labeled c_1, c_2, c_3, c_4 , respectively.

2.3 Flow-cutter

Flow-cutter is developed by the team lead by Prof. Dorothea Wagner from KIT. This heuristic algorithm is a branch of a previously published work, *Graph Bisection with Pareto-Optimization*.⁷

2.4 Other Submissions

There are in total 12 submission to the treewidth problem on PACE-2016 and PACE-2017.⁸ We only chose solvers from the 2017 submissions because according to the report of PACE-2017, all submissions from 2017 significantly outperformed all submissions of PACE-2016.⁹ While we initially intended to test the first two places in both heuristic and exact branch, the exact solvers could only achieve less than 10% of benchmark instances. Thus, we only took the two highest-ranking heuristic solvers from submissions of PACE-2017.

3 Dtrees

A dtree of a CNF is a tuple $(T, \text{clauses})$, where T is a full binary tree whose leaf nodes are in one-to-one corresponding with the CNF clauses, and **clauses** is a mapping from leaf nodes to clauses. Figure 2 depicts an example CNF and two of its dtrees.

Like a tree decomposition, the notion of width is also defined for a dtree. Each node in a dtree has a cluster and the width of the dtree is defined as the size of its largest cluster minus one. To define clusters, we first define the cutsets and contexts for internal nodes. Let N be an internal node in a dtree,

- **vars** is defined as all variables in its clause for a leaf, and **vars** of all children for an internal node. For an internal node N with left child C_l

⁷<https://arxiv.org/abs/1504.03812>

⁸<https://github.com/PACE-challenge/Treewidth>

⁹<https://drops.dagstuhl.de/opus/volltexte/2018/8558/pdf/LIPIcs-IPEC-2017-30.pdf>

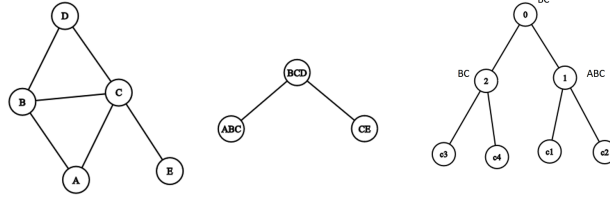


Figure 3: connectivity graph, tree decomposition, and dtree of $(A \vee B) \wedge (A \vee C) \wedge (B \vee C \vee D) \wedge (C \vee E)$. We label the 4 clauses as $c1, c2, c3, c4$ respectively. In the dtree, clusters of each internal node is labeled next to the node.

and right child C_r , we denote $\text{vars}^l(N) := \text{vars}(C_l)$ and $\text{vars}^r(N) := \text{vars}(C_r)$

- $\text{acutset}(N) := \bigcup_P \text{cutset}(P)$, where P is ancestor of N
- $\text{cutset}(N) := (\text{vars}^l(N) \cap \text{vars}^r(N)) \setminus \text{acutset}(N)$
- $\text{context}(N) := \text{vars}(N) \cap \text{acutset}(N)$

The cluster of a leaf dtree node n is $\text{vars}(c)$ where c is the clause attached to node n . The cluster of an internal dtree node n is $\text{cutset}(n) \cup \text{context}(n)$. Figure 2 depicts the cluster of each dtree node. The width of the dtree on the left is 1. The width of the dtree on the right is 2.

There are different methods for constructing dtrees, leading to dtrees with different widths with implications on the complexity for further computations (generally speaking, we want a dtree with the smallest width possible). In c2d, dtrees are generated using hypergraph partitioning. The original CNF is converted into a hypergraph where nodes are variables and hyperedges are clauses. C2d then recursively partitions the hypergraph two-fold. Each partition then forms their own subtree recursively, and the two trees are connected to their common parent.

4 Converting a Tree Decomposition into a Dtree

4.1 Overview

In this project we propose a new way of generating dtree for a CNF, by computing its tree decomposition. First, the CNF is converted into a connectivity graph. A connectivity graph of a CNF is a unique graph whose nodes are CNF variables, and an edge is created for variables sharing a clause. A tree decomposition is then generated for the connectivity graph. Finally, we use the tree decomposition as a heuristic to generate a dtree. Given a tree decomposition of treewidth w , our new method produces a dtree of width $w' \leq w$.

For example, here we have a simple CNF f : $(A \vee B) \wedge (A \vee C) \wedge (B \vee C \vee D) \wedge (C \vee E)$, Figure 3 shows the result of each step discussed above.

Algorithm 1 ClauseInsertion(*tree_decomp*, *clauses*)

```
1: clause_bags  $\leftarrow$  EmptyMap()
2: for Every clause f in clauses do
3:   Create new bag b'
4:   vars(b')  $\leftarrow$  vars(f)
5:   Find an arbitrary bag b in tree_decomp, where vars(b')  $\subseteq$  vars(b)
6:   Create new edge between b and b'
7:   clause_bags[b']  $\leftarrow$  f
8: end for
9: for Every leaf l in leaves(clauses) do
10:  if l  $\notin$  clause_bags then PurgeLeaf(tree_decomp, l)
11:  end if
12: end for
```

Algorithm 2 PurgeLeaf(*tree*, *node*)

```
1: if node is leaf then
2:   Remove node from tree
3:   PurgeLeaf(tree, parent(node))
4: end if
```

4.2 Algorithm

In this section we dive into the subroutines that build up to the conversion algorithm we just described in the previous section.

4.2.1 Algorithm 1: ClauseInsertion

The first step involves inserting clauses into the tree decomposition as leaf nodes. Inputs to this function are the tree decomposition, as a undirected graph and node-to-bag mapping, and a set of clauses. For each clause, it finds a containing bag and makes the clause a new child of that bag. Then all branches of the resulting tree that do not lead to a newly-inserted leaf is removed by calling Algorithm 2 on remaining leaves of the original tree decomposition.

We made two modes of Algorithm 1: 1) "all-clause" mode: inserting all clauses, or 2) "subsuming" mode: inserting a smallest set of subsuming clauses. A clause is subsumed by another when the latter's variables is a superset of the former's. Inserting clauses in subsuming mode requires expanding dtree leaves into subtrees of subsumed clauses.

4.2.2 Algorithm 3: TreeToBinaryTree

Algorithm 3 converts the tree decomposition to a full binary tree. The input to this algorithm is the result from Algorithm 1. The input tree is rooted first, then each of its node is processed based on these cases: if it's a leaf or has exactly two children, it's skipped; if it has more than two children, the node is extended to a full binary tree using Algorithm 4; if it has only one child, it's

Algorithm 3 TreeToBinaryTree(*tree_decomp*)

```
1: Root tree_decomp at arbitrary non-leaf node root
2: stack  $\leftarrow$  [root]
3: while stack is nonempty do
4:   current_node  $\leftarrow$  stack.pop()
5:   for Every child c of current_node do
6:     stack.push(c)
7:   end for
8:   if |children(current_node)| = 2 OR current_node is leaf then
9:     Continue
10:  else if |children(current_node)| > 2 then
11:    ResolveOverflow(tree_decomp, current_node)
12:  else if |children(current_node)| = 1 then
13:    Connect parent(current_node) and child(current_node)
14:    Remove current_node
15:  end if
16: end while
```

Algorithm 4 ResolveOverflow(*tree*, *node*)

```
1: if |children(node)| <= 2 then
2:   Return
3: end if
4: Create new bag extension, vars(extension)  $\leftarrow$  vars(node)
5: Create edge between node and extension
6: child1  $\leftarrow$  children(node)[0]
7: for Every child c  $\neq$  child1 in children(node) do
8:   Disconnect c and node
9:   Create new edge between c and extension
10: end for
11: ResolveOverflow(tree, extension)
```

removed and the child is connected to the parent. Output of Algorithm 3 is a rooted full binary tree whose leaves are in one-to-one correspondence to the clauses of the clauses inserted from Algorithm 1.

4.2.3 Algorithm 5

Finally the full binary tree is converted to a dtree. Input to this algorithm is the directed full binary tree, which is result of Algorithm 3. It uses the tree structure of the modified tree decomposition as the full binary tree of a dtree, and uses the one-to-one correspondence between leaves and clauses to construct the mapping from leaves to clauses. This subroutine outputs the final result, a width-bounded dtree.

Algorithm 5 Bt-2-Dt(*tree_decomp*, *root*, *clauses*)

```
1:  $dt \leftarrow EmptyTree()$ 
2:  $root(dtree) \leftarrow root$ 
3: for Every bag  $b$  in tree_decomp do
4:   Create new dtree node  $n$ 
5:   Connect  $n$  to  $dt$ , topologically equivalent to  $b$  in tree_decomp
6:   if There is clause  $f \in clauses$ , where  $vars(f) = vars(b)$  then
7:      $clause(n) \leftarrow f$ 
8:      $clauses.remove(f)$ 
9:   end if
10: end for
11: Return  $dt$ 
```

5 Evaluation

We chose the public instances of Model Counting Competition 2021 and 2022 as our benchmark instances. The test bench is constructed as follows: 1) a moral graph is generated for the input CNF and saved in memory; 2) tree decomposition solver invoked, moral graph is copied to its standard input; the solver outputs its result to standard output; 3) a dtree is generated using the tree decomposition result, using both all-clause mode and subsuming mode; 4) the dtree is copied to a file, and fed into c2d to generate a smooth d-DNNF and perform model counting.

We manually set a timeout for some of the steps. Steps 2 and 3 are limited to 5 minutes in total, and step 4 is limited to 25 minutes in total. For baseline, we ran each benchmark instance using c2d using hypergraph partitioning as the dtree generation method, with 30 minutes time limit. All test instances are run on a CentOS Linux machine with 256 GB RAM and 32 Intel Xeon CPU E5-2670 2.60GHz CPUs.

In the following, we use c2d_v to refer to the original c2d which constructs dtrees using hypergraph partitioning. We also use c2d_f to refer to c2d based on dtrees constructed using flow-cutter. Finally, we use c2d_t to refer to c2d based on dtrees constructed using Tamaki-2017. Note that c2d_f and c2d_t can run in two modes, depending on whether we include all clauses in the constructed dtree or only the subsuming ones.

5.1 Results

We will discuss the results using the following metrics: success rates of finishing the compilation and model count, d-DNNF circuit size, and d-DNNF compilation time. While the first one applies to all input instances, the latter two only applies to instances where both the solver/mode in question and baseline solved.

5.1.1 Success Rates

Both solvers on both modes succeeded in cases c2d_v failed and failed in some that c2d_v succeeded. Specifically, for c2d_f, all-clause mode did 10 extra cases and failed 17 extra cases than c2d_v; subsuming mode got 9 extra success and 15 extra fail than c2d_v. For c2d_t, all-clause mode succeeded 11 extra cases and failed 17 extra cases than c2d_v, while subsuming mode succeeded 12 extra cases and failed 19 extra cases than c2d_v. Both solvers, in total, solved slightly less instances in the 198 benchmarks than c2d_v, and the results are very similar.

5.1.2 d-DNNF Size

The size of generated d-DNNF measures the succinctness of the new system’s outputs. On the contrary to success rates, both c2d_f and c2d_t improves significantly over c2d_v in d-DNNF size in both modes, and we can see non-trivial performance difference across solvers. c2d_t was able to achieve on average 4.8x more space efficiency, where all-clause and subsuming modes’ averages differ by 0.075; c2d_f achieved an average of 2.9x space shrink, where subsuming mode has 1.108x over all-clause mode’s average.

5.1.3 d-DNNF compilation time

While the new systems take longer than c2d_v to compile a dtree, due to allowing 5 minutes for each solver on all instances, we have observed significant improvements over d-DNNF compile times. On average, c2d_t achieved 4.6x speed up using all-clause mode, and 5.11x speed up using subsuming mode. c2d_f achieved average 3.53x using all-clause mode and 3.10x using subsuming mode.

6 Conclusion

Through evaluation data presented in Section 5, we have shown that constructing dtree with our proposed new method improves significantly on c2d_v in terms of d-DNNF compilation speed and d-DNNF compiled size. Admittedly, the new systems solved less benchmarks than c2d_v among all 198 public instances of Model Counting Competition 2021 and 2022; however, the difference is less than 5% of all instances. Among all 4 solver-mode combinations, we have also observed significant difference in performance. According to the statistics mentioned above, c2d_t on average has outperformed c2d_f in both d-DNNF size and speed. While modes doesn’t seem to affect c2d_t’s space performance, there’s a 0.37x speedup improvement when switching from all-clause mode to subsuming mode. Therefore, c2d_v with subsuming mode is the more preferable option to construct dtree with for our new method. It has exhibited better speed and circuit size in end results while performs similarly to the other 3 options in terms of success rate.